

Capacity & Demand Intelligence — Project Mastery Guide

This document is a personal learning and interview guide. It reverse-engineers the repository as it exists. It is not a README and it is not a claim about Chin Hin’s real operations.

Status labels used throughout

Label	Meaning
IMPLEMENTED	The code does this, and a user can see or call it.
PARTIALLY IMPLEMENTED	A piece exists, but it is narrower than the business idea.
PLANNED / CONCEPTUAL	Discussed as a future fit. Not built.
ASSUMED	A number or rule the prototype needs, marked as an assumption in code.
SYNTHETIC DATA	Invented for the prototype. Not an extract from Chin Hin, an ERP, Excel, or a customer.

The repository never mentions the name Chin Hin. Plant names, customers, projects, volumes, margins, penalties, and delay costs are written in `backend/app/seed.py`. **This is synthetic data created for the prototype and does not represent actual Chin Hin operational data.**

There is no delivery table, no actual-production table, no separate project-schedule table, no authentication, and no automated test suite. Those absences are stated where they matter. Do not describe them as features.

1. PROJECT OVERVIEW

Project name. Capacity & Demand Intelligence.

Business context. A building-materials plant makes products that cannot be swapped for each other, on lines that can only produce so many cubic metres on a given day. The prototype books two plants and three products for 1–30 October 2026.

Industry framing in the code. Ready-mix concrete (Grade 40, Grade 50) and precast wall panels, measured in m³, money in RM. Source: `backend/app/seed.py`, products G40, G50, PCS.

Main problem. When orders due by a date need more than that plant can supply by that date, who should receive the cubic metres, and what margin, penalty, and programme delay does the choice leave behind?

Main objective, as coded. Minimise the expected ringgit consequence of unserved quantity. Source: `backend/app/engine.py`, function `_solve_lp`, and the string returned as `objective` from `solve_bucket`.

Intended users, as the screens imply. A planner who reviews a recommendation, edits quantities, and records a named decision. The sidebar field “Planner on the audit trail” defaults to A. Rahman and is stored in the browser (`frontend/src/planner.tsx`). That name is a UI default, not a real employee record.

Expected business value, as the app measures it. A lower expected consequence than three simple rules on the same capacity: earliest required date, internal projects first, and external customers first. The difference versus earliest-date is labelled “value protected”. It is a **modelled** difference on synthetic orders. It is not observed savings at a real company.

What the prototype currently does.

- Loads a synthetic book into SQLite on startup (`backend/app/seed.py`, `seed`).
- Shows one combined demand book and can draft a new line from pasted text (`backend/app/extractor.py`).
- Computes dated supply and highlights dates where cumulative demand runs ahead of cumulative supply (`capacity_view`).
- Solves a linear programme separately for each plant × product (`solve_bucket`, `_solve_lp`).
- Compares that solution with three greedy rules (`_greedy`).
- Lets a person edit the allocation, rejects edits that cannot be produced by the required date (`feasibility`), and stores the recommendation versus the recorded decision (`POST /api/decisions`).
- Re-runs the same engine under scenario edits (`apply_scenario`, `POST /api/scenarios/compare`).
- Compares earliest-date, the recommendation, and any recorded human decision on a Business Impact page (`backend/app/impact.py`).

30-second explanation

The plant can only make so much of one product by a required date. Internal projects and external customers compete for those cubic metres. The system prices the cost of leaving each order short — margin, contractual penalty, and programme delay, weighted by how firm the order is — and recommends the allocation that leaves the smallest expected cost. A planner approves or changes it, and that decision is stored. The October 2026 book is synthetic.

1-minute explanation

A month can look as if it has enough capacity while a particular week does not. Later production cannot serve a pour whose date has passed. This prototype puts internal projects and external customers in one demand table, computes available capacity as daily capacity minus production already committed outside the book, and adds only inventory above safety stock. For each plant and product it solves a linear programme: decide how many cubic metres of each order to leave unserved, without exceeding dated capacity or usable inventory, and without using a day after the required date. It then shows what three simpler rules would have done on the same supply. The planner can override the quantities. If the override cannot physically be made by the due dates, it is rejected. If it can, the reason, the time, and the planner name are written to a decisions table. Nothing in the repository is a Chin Hin data extract.

3-minute explanation

Start from the decision, not the screens. Someone has to choose which orders absorb a shortage. Serving the earliest date, the internal project, the external customer, or the highest margin can each be the wrong rule once penalties and delay costs differ, and once an order can only use capacity on or before its own date.

The data is one SQLite file, `backend/data/cdi.db`, created by `seed()`. Plants, products, a 30-day capacity calendar, one inventory snapshot per plant-product, and 18 demand lines are inserted from Python constants. **Synthetic**. On startup, `seed()` reloads that book only if the stored seed version is not `2026-10-hero-1`.

Demand is not split into two systems. `demand_type` is `Internal` or `External` on the same demands row. Each row already carries its money: contribution margin, contractual penalty, delay days, and delay cost per day, all as totals for the full requested quantity. Confidence is `Confirmed`, `Probable`, or `Forecast`.

Available capacity on a day is `daily_capacity - planned_production`, floored at zero. Usable inventory is `max(0, on_hand - safety_stock)`. Supply that can serve an order is usable inventory plus available capacity on days on or before the required date, and only at that order's plant and product. Products are not substitutes. Plants are not balanced against each other. That is because `allocate()` calls `solve_bucket()` once per plant-product pair. There is no variable that moves Grade 40 from Shah Alam to Pasir Gudang.

Inside a bucket, PuLP builds a minimisation problem and CBC solves it (`_solve_lp`). The cost of one unserved cubic metre is the order's expected consequence divided by requested quantity. A very small extra term prefers using inventory before production. It is too small to change which order is served. Partial fulfilment is allowed: margin, penalty, and delay scale with the unserved fraction. Programme days are counted only for internal demand (`consequence_for_unserved`).

The same supply is also scheduled by three greedy policies so the planner can see the gap. Explanations are written in `_reasons` and `_narrative`. An expedite table compares emergency cost per m³ with the consequence of the shortfall. Emergency cost is an assumption and is not added to base capacity.

The UI is six pages. Control Tower states the question and the hotspots. Demand Hub lists and filters the book and can add a drafted line. Capacity Planner shows the cumulative gap. Allocation Decision is where the recommendation is reviewed, edited, and recorded. Scenario Simulator reruns up to three cases. Business Impact compares earliest-date, the linear programme, and the latest human decision. Recording a decision does not release a production order. It stores an audit row. The next optimisation still starts from the demand book and the calendar, not from the approved quantities.

2. BUSINESS PROBLEM

IMPLEMENTED problem statement. The Control Tower returns this question from `control_tower()`:

When plant capacity is constrained and total demand exceeds available capacity, how should the available capacity be allocated between internal projects and external customers, and what is the financial and operational consequence of that decision?

The sharper version in the same function's `insight` text: adding the month together can show a surplus, while required dates leave a dated shortfall. The decision is which orders absorb that shortfall.

Capacity constraints. Each plant-product has a daily capacity. Part of it is already called `planned_production` and is not available to this book. Maintenance notes cut specific days (Shah Alam Grade 40 on 7–8 Oct 2026; Pasir Gudang Grade 50 on 14–15 Oct 2026). Source: `_capacity_for` in `seed.py`.

Internal demand. Rows with `demand_type = 'Internal'`. In the seed they are group projects or the plant's own silo work. If they are missed, the coded consequence is programme delay: `delay_days_if_unserved × delay_cost_per_day`, plus any contribution margin on the row.

External demand. Rows with `demand_type = 'External'`. In the seed they look like customer orders. If they are missed, the coded consequence is contribution margin plus contractual penalty. External rows in the seed have `delay_days_if_unserved = 0` and criticality n/a. Programme days are forced to zero for non-internal rows even if delay days were set (`consequence_for_unserved`).

Demand planning. PARTIALLY IMPLEMENTED. The book can hold Confirmed, Probable, and Forecast lines. There is no statistical forecast model. Forecast rows are typed in by the seed, or parsed from text that contains the word "forecast".

Supply limitations. Supply is plant-specific, product-specific, and date-specific. A cubic metre after the required date is stranded for that order. `stranded_capacity_m3` in `solve_bucket` sums available capacity after the latest date that still has an unserved order.

Inventory. One opening balance per plant-product, dated at the horizon start. Safety stock is reserved. Only the surplus is usable.

Production constraints. The model can assign production of an order only to days on or before its required date, and only up to that day's available capacity. It does not model changeovers, crews, curing time, or dispatch slots. Those are not in the schema.

Competing priorities. The optimiser has no internal-versus-external weight. Competition is the expected ringgit per unserved cubic metre, subject to the date window.

Programme delay. For internal lines, unserved fraction × delay days. A half-served order is treated as half the delay days. That is a modelling choice in `consequence_for_unserved`, not a claim about how a real programme actually slips.

Margin. `contribution_margin` is the total RM for the full requested quantity, not RM per m³. The engine divides by quantity when it needs a unit rate (`line_economics`).

Working capital. ASSUMED. `build_impact` sets working capital equal to on-hand × inventory unit value, with an explicit note that this stands in for stock that is paid and not yet billed. The unit values are assumptions on the product row.

Lost sales / unmet demand. Unserved m³, and the margin, penalty, and delay cost on that fraction. The app does not model a lost customer relationship beyond the penalty stored on the row.

What happens if demand is greater than available capacity?

Two different comparisons exist. Do not mix them up.

1. **Month total.** `horizon_surplus_m3 = max(0, total available supply - total demand)` and `aggregate_gap_m3 = max(0, total demand - total available supply)`, summed across buckets in `allocate()`. A surplus here means the month's cubic metres are enough if dates did not matter.

2. **Dated supply.** An order may only consume inventory plus capacity on or before its required date. `shortfall_m3` on a bucket is the unserved quantity after the linear programme. Control Tower shows this as “Capacity gap”. The code’s own insight says the month can be in surplus while the dated shortfall is still large.

If dated demand exceeds dated supply, some requested quantity stays unserved. The engine chooses which quantity, by minimising expected consequence. It does not create extra capacity. The expedite screen only advises whether paying an assumed emergency cost would be cheaper than accepting the shortfall.

Why this is a decision problem, not only a dashboard

A dashboard can show that a date is short. It does not choose the orders. This repository does four things a chart does not:

- It computes a feasible assignment (`_solve_lp`).
- It prices that assignment against three alternative rules (`_greedy`).
- It blocks an infeasible human edit (`feasibility`).
- It stores the recommended quantities and the recorded quantities (`decisions`).

The charts exist to explain that decision. They are not the product.

Why the simple rules fail

All four statements below are demonstrated in code, not just as opinions. `_greedy` implements the rules. `_solve_lp` implements the recommendation. `selfcheck.py` asserts that, on the Shah Alam Grade 40 book, the optimised expected consequence is lower than earliest-date, internal-first, and external-first.

Highest margin first. Margin is only one term. JKR School Cluster has a small margin (RM5,400 on 180 m³) but a RM22,000 penalty. Merdeka Podium’s margin is RM16,000 on 400 m³, but the large term is 3 days × RM20,000. Ranking by margin alone drops the penalty and the delay. The objective uses margin + penalty + delay cost, then divides by quantity, then multiplies by confidence.

Earliest order first. JKR is due 6 Oct, before Merdeka and Gamuda on 9 Oct. Earliness decides which days can produce the order. It does not decide that the earlier order has the larger consequence. The narrative in `_narrative` says this in words: “Earliness is a production constraint, not a priority rule.” On this synthetic book the earliest-date rule leaves a higher expected consequence than the linear programme.

Internal projects first. Merdeka is internal and critical, but Gamuda’s confirmed penalty is real money too. An internal-first sort (`policy == "internal"` in `_greedy`) serves internal rows before external rows, using unit expected consequence only as a tie-break inside the group. That can spend scarce cubic metres on a weaker internal line and leave a larger external penalty. The optimiser refuses that preference.

External customers first. The mirror image. A large external order can consume the window and push a critical internal pour past its date. External-first is also a sort, not a second optimisation. Because it walks orders one by one and consumes earlier days first, a later external order can take capacity that an earlier external order cannot use. That is why a mental “serve the best external customers” story will not match `_greedy` exactly. In an interview, call these three policies priority rules on the same capacity, not alternative linear programmes.

3. BUSINESS PROCESS

The chain below is what the software actually walks. "Production / delivery" after approval is **not executed**. The process stops at a stored decision.

Demand

What happens. Eighteen seed lines, plus any line a person adds, sit in demands.

Data. Customer or project name, type, plant, product, required date, quantities, confidence, margin, penalty, criticality, delay days, delay cost per day, source, notes.

Who. A demand planner or the person pasting a PO or email into Demand Hub.

Why. Allocation cannot start until competing orders are in one list.

Demand consolidation

IMPLEMENTED, narrowly. Internal and external rows are already one table. Filters on GET `/api/demands` narrow the view. They do not merge duplicates. Extraction (`extract_demand`) drafts one new row from text. Saving it (POST `/api/demands`) inserts it. Extraction does not allocate.

Capacity

What happens. For every plant, product, and day in October 2026, the seed writes `daily_capacity` and `planned_production`.

Data. `capacity_calendar`.

Who. In a real plant this would be industrial engineering or the planner who owns the line plan. Here it is generated by `_capacity_for`.

Why. The shortage is daily, not annual.

Inventory

What happens. One snapshot: on-hand and safety stock at 2026-10-01.

Who. Warehouse or the materials controller. Here, constants in `INVENTORY`.

Why. Stock already on the floor can cover a pour without using a production day. Safety stock is not available to give away.

Supply availability

Formula, from `capacity_view`'s formulas dict and from `load_world`.

- Available capacity on a day = $\max(0, \text{daily capacity} - \text{planned production})$
- Usable inventory = $\max(0, \text{on-hand} - \text{safety stock})$
- Available supply for the month = sum of available capacity + usable inventory
- Dated supply through a date = usable inventory + available capacity on days on or before that date

Constraint detection

What happens. `capacity_view` marks a date constrained when cumulative demand due through that date exceeds cumulative supply through that date. `solve_bucket` marks a bucket constrained when unserved quantity after optimisation is above 0.05 m^3 (TOL).

Who. The planner, on Capacity Planner and Control Tower.

Why. This is the moment the allocation engine matters. If the bucket is not short, the orders are not competing.

Allocation optimisation

What happens. `allocate()` → `solve_bucket()` → `_solve_lp()` for every plant × product, including empty ones.

Who. The system. The planner does not pick the solver.

Why. The feasible assignments are too entangled to price by eye once several orders share a window.

Recommendation

What happens. Each served and unserved line gets a plain-language reason (`_reasons`), a card (`_cards`), and a comparison with the three rules (`_narrative`).

Human review

What happens. Allocation Decision shows the reason, the RM per m³, and an editable quantity. After a short pause the page calls `POST /api/allocate/override`.

Approval / override

What happens. `POST /api/decisions` checks feasibility. If quantities differ from the recommendation by more than 0.1 m³, status is `modified` and the reason must be at least 8 characters. Otherwise status is `approved`. Both the recommended JSON and the final JSON are stored.

Who. The named planner.

Why. The model is not allowed to release the plan by itself. Source comment in `economics.py`: "The model recommends. A person approves or overrides, and that decision is stored."

Production / delivery

NOT IMPLEMENTED. No route updates `planned_production`. No delivery date is written back. No pick list is created. Say this plainly in an interview: the prototype ends at the decision record.

Business outcome

What happens. `build_impact()` prices earliest-date, the linear programme, and the latest matching human decision. Inventory value, excess versus safety stock and 14-day demand, and the expedite screen are reported beside that comparison. All money figures are calculated from synthetic inputs and labelled assumptions where the code says so.

4. ACTUAL PROJECT ARCHITECTURE



frontend/vite.config.ts proxies /api to http://127.0.0.1:8000. The frontend does not contain the optimiser.

Component	Path	Purpose	Inputs	Processing	Outputs	Why it exists
HTTP API	backend/app/main.py	Routes, validation, startup seed	JSON bodies, query params	Pydantic checks, then calls engine or SQL	JSON	Keeps the solver off the browser
Schema and SQL	backend/app/db.py	Tables and connection	None at runtime beyond the file path	CREATE TABLE IF NOT EXISTS, fetch_all	cdi.db	One local database, no server install
Economics	backend/app/economics.py	Shares money math	One demand dict, an unserved quantity	Margins, penalties, delay, confidence	Unit rates and scaled consequences	So the solver, the scenarios, and the impact page use one definition
Seed	backend/app/seed.py	Synthetic book	Constants	Inserts plants, products, calendar, inventory, demands	Rows, or a no-op if version matches	The prototype must open with a constrained case
Engine	backend/app/engine.py	Simple LP, greedy rules, explanations, feasibility	World from SQLite, optional scenario	CBC minimise, then narratives	Buckets, totals, reasons	This is the decision
Extractor	backend/app/extractor.py	Text to draft	Raw string, plant list, product list	Regular expressions	Draft JSON, steps, warnings	Shows where document intake would sit. It does not decide allocation
Impact	backend/app/impact.py	Before and after stock value	Baseline allocation, decisions table	Sums policies, overlays latest decision per plant-product	Impact JSON	Separates "what the rule would do" from "what a person recorded"
Self-check	backend/app/self-check.py	One scripted assertion of the hero book	Forces reseed, calls allocate	assert	Prints SELF-CHECK PASSED or crashes	Not a test framework. A sanity check
Shell	frontend/src/App.tsx, Layout.tsx, planner.tsx	Sites routes and the planner name	Browser URL, localStorage	React Router	Pages	Navigation and the name written onto decisions
API client	frontend/src/api.ts	Fetch wrapper	Path, JSON	Throws on non-OK	Parsed JSON	One error path
Pages	frontend/src/pages/*	The six screens	API payloads	Display, edits	User actions back to the API	The planner's workflow
Types and format	frontend/src/types/format.ts	Shapes, and RM / m³ / date display	Numbers and ISO dates	Formatting	Strings	Presentation only

Component	Path	Purpose	Inputs	Processing	Outputs	Why it exists
Proxy	frontend/vite.config.ts	Configuring	Browser calls to /api	Proxy	FastAPI	The UI and API can run as two processes

Files with no business role: frontend/src/vite-env.d.ts, frontend/src/main.tsx (mounts React), backend/app/__init__.py (empty package marker).

Not in the repository. Services layer, ORM models, authentication, a message queue, Power BI, Power Automate, SharePoint, ERP connectors. Do not draw them as built components. Section 20 says where they could sit later.

5. DATA USED

Every dataset below is **SYNTHETIC** unless a person types a new demand line or a decision during a demo. A line typed in the UI is still not Chin Hin data. It is data entered into the prototype.

There is **no** separate table for external orders, internal projects, delivery, actual production, or a project schedule. External orders and internal projects are rows in `demands`. "Project schedule" and "Customer PO" are values in the source column. Planned production is a column on `capacity_calendar`, not its own table.

meta

- **Purpose.** Remembers whether this database already has the current seed.
- **One row.** One key/value.
- **Primary key.** `key`.
- **Columns.** `key` TEXT, `value` TEXT.
- **Example.** `seed_version = 2026-10-hero-1`. Also `horizon_start`, `horizon_end`.
- **Created.** `set_meta` inside `seed()`.
- **Consumed.** `seed()` skips the reload when the version matches.
- **SYNTHETIC.**

plants

- **Purpose.** The two works.
- **One row.** One plant.
- **Primary key.** `id`. Unique `code`.
- **Columns.** `id`, `code`, `name`, `location`.
- **Example.** (1, 'SA', 'Shah Alam Works', 'Selangor'), (2, 'PG', 'Pasir Gudang Works', 'Johor').
- **Created.** PLANTS in `seed.py`.
- **Consumed.** Every demand, calendar row, and inventory row points at `plant_id`.
- **SYNTHETIC.** These are not evidence of real plant capacity.

products

- **Purpose.** What the plant makes, plus two money assumptions used outside the base solver.
- **One row.** One product.
- **Primary key.** `id`. Unique `code`.
- **Columns.** `code`, `name`, `unit`, `inventory_value_per_m3`, `emergency_cost_per_m3`, `emergency_cost_is_assumption`.
- **Example.** G40 Ready-Mix Grade 40, unit m³, inventory value RM280/m³, emergency RM95/m³, assumption flag 1. G50 is RM340 and RM110. PCS is RM1,200 and RM180.
- **Created.** PRODUCTS. The insert always sets `emergency_cost_is_assumption = 1`.
- **Consumed.** Emergency cost is used only by `expedite_advice`. Inventory value is used by Control Tower and Business Impact. The linear programme does not put either number in its objective.
- **ASSUMED** unit values. **SYNTHETIC** product list.

capacity_calendar

- **Purpose.** Daily theoretical capacity and the slice already committed outside this book.
- **One row.** One plant, one product, one date.
- **Primary key.** `id`. Unique (`plant_id`, `product_id`, `prod_date`).
- **Columns.** `daily_capacity` REAL, `planned_production` REAL, `note` TEXT, foreign keys to plants and products.
- **Example.** Shah Alam G40 on a normal weekday: capacity 180, planned 80, so available 100. On 7 Oct 2026: capacity 100, planned 60, note "Line 2 maintenance. Available capacity cut to 40 m³."

- **Created.** `_capacity_for` × 2 plants × 3 products × 30 days.
- **Consumed.** `load_world` derives `available_capacity`. The solver's capacity constraint uses that derived number, not `daily_capacity` directly.
- **SYNTHETIC.** Weekend and maintenance patterns are hand-written so the hero date is short.

inventory

- **Purpose.** Opening stock and the safety-stock reserve.
- **One row.** One plant-product snapshot. Not a daily stock history. Unique (`plant_id`, `product_id`).
- **Primary key.** `id`.
- **Columns.** `as_of_date`, `on_hand`, `safety_stock`. `usable` is not stored. It is computed.
- **Example.** Shah Alam G40: on-hand 145, safety 80, so usable 65, as of 2026-10-01.
- **Created.** `INVENTORY`.
- **Consumed.** Inventory constraint in `_solve_1p`. Capacity charts. Impact valuation.
- **SYNTHETIC.**

demands

- **Purpose.** The single order book. Internal and external.
- **One row.** One order line for one plant and one product.
- **Primary key.** `id`. Unique `demand_code`.
- **Important columns.**

Column	Type	Role	Example
<code>demand_code</code>	TEXT	Business key	INT-MERDEKA
<code>demand_type</code>	TEXT	Internal or External	Internal
<code>customer_or_project</code>	TEXT	Display name	Merdeka Podium
<code>customer_type</code>	TEXT	Label only	Group project
<code>plant_id, product_id</code>	INTEGER	Where it can be made	1, 1
<code>required_date</code>	TEXT YYYY-MM-DD	Last day that can serve it	2026-10-09
<code>requested_quantity</code>	REAL	What the solver tries to cover	400
<code>confirmed_quantity</code>	REAL	Stored and shown. Not used by the solver	400
<code>demand_status</code>	TEXT	Firm / Open / Forecast. Not used by the solver	Firm
<code>confidence_level</code>	TEXT	Confirmed, Probable, Forecast. This is what the solver weights	Confirmed
<code>contribution_margin</code>	REAL	Total RM if the full quantity is served	16000
<code>contractual_penalty</code>	REAL	Total RM if the full quantity is missed	0 on Merdeka, 30000 on Gamuda
<code>project_criticality</code>	TEXT	Shown to the planner. Not a solver coefficient , unless a scenario changes it and thereby rescales delay cost	Critical
<code>delay_days_if_unserved</code>	REAL	Days if the whole order is missed	3
<code>delay_cost_per_day</code>	REAL	RM per day for a full miss	20000

Column	Type	Role	Example
source	TEXT	Where the row claims to come from	Project schedule, Customer PO, Sales forecast, Manual, Email intake, Simulated OCR
notes	TEXT	The story written for the demo	Free text
created_at	TEXT	UTC timestamp	Set at insert

- **Created.** `_demand_rows()` for the original 18. `POST /api/demands` for new lines, coded INT-IN-001 or EXT-IN-001.
- **Consumed.** `line_economics`, `_solve_lp`, the Demand Hub table.
- **SYNTHETIC** for the 18. Names such as Gamuda, JKR, Sunway, YTL, IJM, WCT, SP Setia, MRCB, and ECRL are labels in a made-up book. They are not those organisations' orders.

`customer_type` is not read by the optimiser.

decisions

- **Purpose.** Audit trail. Recommendation versus what the person recorded.
- **One row.** One approve or modify action for one plant-product at one time.
- **Primary key.** `id`. There is **no** foreign key to `plants` or `products`. Names are copied onto the row.
- **Columns.** `created_at`, `username`, `plant_id`, `product_id`, `plant_name`, `product_name`, `recommended_json`, `final_json`, `override_reason`, `status` (approved OR modified), `consequence_recommended`, `consequence_final`, `unserved_recommended`, `unserved_final`.
- **Example.** Empty until someone clicks "Record decision".
- **Created.** `record_decision` in `main.py`.
- **Consumed.** Control Tower (latest 8), Allocation audit table, `build_impact` (latest row per plant-product, and only if the demand ids still match).
- A recorded decision is **actual relative to the demo session**. The quantities it compares are still priced from synthetic economics.

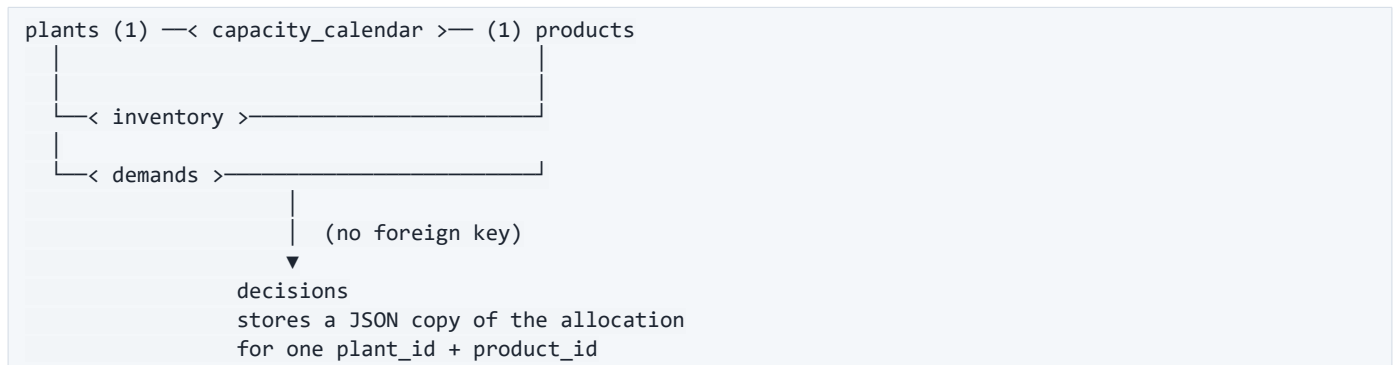
Scenario data

Not a table. A scenario is a JSON body: `name`, `capacity_factor` from 0.5 to 1.5, optional `plant_id` and `product_id`, `inventory_adjustments`, `demand_adjustments`. `apply_scenario` copies the in-memory world and edits the copy. The database is unchanged. Source: `ScenarioIn` in `main.py`, `apply_scenario` in `engine.py`.

What was asked for but does not exist

Name in the brief	What the repo actually has
External orders dataset	Rows in <code>demands</code> with type External
Internal projects dataset	Rows in <code>demands</code> with type Internal
Production dataset	<code>planned_production</code> on the calendar only. No actual production
Delivery dataset	Nothing
Project schedule dataset	<code>source = 'Project schedule'</code> on some demand rows, plus delay fields on the same row
Financial assumptions dataset	Product columns, demand money columns, and the <code>ASSUMPTIONS</code> list in <code>economics.py</code>

6. DATA MODEL



Relationships that are enforced.

- `capacity_calendar.plant_id → plants.id, product_id → products.id`. Many calendar rows per plant and per product. One row per plant-product-date.
- `inventory.plant_id → plants.id, product_id → products.id`. One inventory row per plant-product.
- `demands.plant_id → plants.id, product_id → products.id`. Many demands per plant-product.
- `PRAGMA foreign_keys = ON in connect()`.

Relationships that are not enforced.

- `decisions.plant_id` and `product_id` are plain integers. Deleting a plant would not be blocked by this table. The prototype never deletes plants.
- A decision does not point at `demands.id` with a foreign key. The link is inside JSON. `build_impact` checks that the set of demand ids still matches. If someone adds or deletes a line after the decision, that bucket falls back to the recommendation and a note is added.

How internal and external demand are unified. One table, one discriminator column, one solver. There is no union query. `demand_type` changes two things only: programme days are counted for Internal, and the greedy policies sort Internal or External first. The linear programme does not read `demand_type`.

How demand connects to plant and product. Foreign keys. The solver never looks across them. Grade 50 demand cannot consume Grade 40 capacity.

How demand connects to inventory and capacity. Not by a foreign key from `demands` to those tables. The connection is computed: same `plant_id` and `product_id`, then dates. If two rows shared a plant and product they share one inventory pool and one calendar. That is intentional.

How a project schedule connects to internal demand. It does not. Delay days and delay cost are columns on the demand row. There is no schedule activity, predecessor, or float.

Why dates matter. `required_date` deletes production variables for later days (if `day <= demand["required_date"]`). `prod_date` is the capacity bucket. Without dates, the month surplus would hide the shortage.

Duplicate counting.

- Two demand rows for the same physical pour would both enter the objective and both consume capacity. Nothing detects that. `demand_code` uniqueness stops the same code, not the same project under two codes.
- Capacity is not double-counted across products: each calendar row is one product.
- Impact sums buckets. Because buckets do not share supply, adding their unserved m^3 does not double-count cubic metres. Adding programme days across projects is a sum of scaled delay days, which can sound like one programme if you are careless. It is a sum of line-level day impacts.
- `confirmed_quantity` is not added on top of `requested_quantity`. Only requested quantity enters the solver. Showing both in the grid is not double-counting in the maths, but a viewer can still misread them as two volumes.

If one demand record appears twice. The second insert with the same `demand_code` fails the unique constraint. A second insert with a new code is treated as another order and will compete for the same cubic metres.

How you would validate this model.

- Every demand's plant and product exist.
 - `confirmed_quantity <= requested_quantity` (the API enforces this on create; the seed is hand-written).
 - `planned_production <= daily_capacity` (the seed happens to obey this; there is no check).
 - `safety_stock <= on_hand` is not required, because usable inventory floors at zero.
 - Required dates fall inside 2026-10-01 to 2026-10-30 (`parse_user_date` for user input).
 - Sum of allocations in a recorded decision does not exceed dated supply (*feasibility*).
 - Re-run `python -m app.selfcheck` after any change to the hero numbers.
-

7. DATA PREPARATION

There is no pandas pipeline and no data-quality job. Preparation is the seed, a few guards, and the regex extractor.

Step	Input	Transformation	Output	Source
Seed gate	Existing <code>meta.seed_version</code>	If it equals <code>2026-10-hero-1</code> and <code>force</code> is false, stop	Unchanged database	<code>seed()</code>
Reseed	Constants	<code>reset_data</code> deletes decisions, demands, inventory, calendar, products, plants, meta, then inserts again	Fresh synthetic book. User decisions are wiped on reseed	<code>reset_data, seed(force=True)</code>
Available capacity	<code>daily_capacity, planned_production</code>	<code>max(0, daily - planned)</code>	<code>available_capacity</code> on the in-memory row	<code>load_world</code>
Usable inventory	<code>on_hand, safety_stock</code>	<code>max(0, on_hand - safety)</code>	<code>usable</code>	<code>load_world</code>
Drop empty orders	Demand rows	Skip <code>requested_quantity <= 0</code>	Solver input	<code>_prepare_demands</code>
Economics	Money columns and confidence	See section 9	Unit expected RM	<code>line_economics</code>
Date parsing for intake	Free text	ISO, 9 Oct 2026, or d/m/yyyy	YYYY-MM-DD or a warning	<code>extractor._date</code>
Quantity parsing	Free text	First number followed by <code>m3</code> or <code>m³</code>	Float or a warning	<code>extractor._quantity</code>
Money parsing	Free text	Label, then RM and a number within 50 characters	Float, or 0 with a warning for margin	<code>extractor._money</code>
Classification	Free text	Keyword rules, not a model	Internal/External, confidence, plant, product	<code>extract_demand</code>
New demand code	Count of existing <code>INT-IN-</code> or <code>EXT-IN-</code> codes	Next sequence	<code>INT-IN-003</code> style	<code>create_demand</code>
Scenario copy	World dict	<code>deepcopy</code> , then multiply capacity, shift stock, patch one demand	In-memory world. Database unchanged	<code>apply_scenario</code>
Snap	Solver floats	If within 0.05 m³ of zero or of full quantity, snap to 0 or full	Clean 400 instead of 399.97	<code>_snap</code>
Rounding	Floats	2 decimal places for m³ and RM	Display and stored totals	<code>round_m3, round_rm</code>

Missing values. The extractor sets missing margin and penalty to 0 and adds a warning. A missing plant, product, quantity, or in-horizon date makes `ok` false, so the UI asks the person to finish the draft. The solver does not impute. Seed rows are complete.

Duplicates. Not merged. Unique `demand_code` only.

Product names. Not standardised from free text beyond keyword match: "grade 40", "g40", "grade 50", "g50", "precast", "wall panel", "facade panel".

Units. The solver assumes m³. The extractor only recognises `m3` or `m³`. A quantity in tonnes would be missed or, if someone typed it into the number field, treated as m³. There is no conversion.

Unrealistic capacity. Not validated. A scenario factor is limited to 0.5–1.5 by Pydantic (`ScenarioIn`). That is the only range check on capacity.

Synthetic generation. Weekday rules in `_capacity_for`, plus two maintenance windows, plus 18 hand-built orders. The hero book is constructed so that demand due by 9 Oct 2026 at Shah Alam Grade 40 is 1,410 m³ and dated supply

is 700 m³. The comment at the top of `seed.py` says this directly.

8. CAPACITY MODEL

Simple meaning

Plant capacity is how many cubic metres the line can make that day. Some of those cubic metres are already promised to work that is outside this order book. What is left can be allocated. Stock above the safety minimum can be allocated too. Stock at or below safety cannot.

Formulas actually implemented

From `load_world` and the `formulas` dict in `capacity_view`:

```
available_capacity(day) = max(0, daily_capacity(day) - planned_production(day))

usable_inventory = max(0, on_hand - safety_stock)

available_supply (month) =  $\Sigma$  available_capacity + usable_inventory

supply_to_date(D) = usable_inventory +  $\Sigma$  available_capacity(day) for day  $\leq$  D

demand_to_date(D) =  $\Sigma$  requested_quantity for orders with required_date  $\leq$  D

gap(D) = max(0, demand_to_date(D) - supply_to_date(D))
```

`shortfall_m3` is not the same as `gap`. `Gap` is a cumulative check at each due date. `Shortfall` is the sum of unserved quantities after the linear programme. They match in spirit on the hero book because the crunch is one shared window. Do not tell an interviewer they are the same variable.

There is no actual production. `planned_production` is an input, not a measured output. Nothing in the app records what was made yesterday.

Maintenance. It is not a separate calculation. `_capacity_for` returns a lower capacity and a higher committed slice on the maintenance dates, and a note. The note is displayed. The maths only sees the resulting available capacity.

Downtime. Not modelled except where the seed author reduced those days. There is no random breakdown.

Utilisation. PARTIALLY IMPLEMENTED. `solve_bucket` builds `binding_dates`: a day is binding when available capacity is above tolerance and used production is within 0.05 m³ of available capacity. There is no utilisation percentage KPI on Control Tower.

Inventory and supply. Usable inventory is added once, at the start of the horizon. It can serve any order in that plant-product, but the balance constraint still cannot use inventory that another order has taken. The solver decides who gets it. A tiny objective weight of 1e-4 on production variables prefers drawing inventory first. `economics.py` states that this weight cannot change who is served, because it is far smaller than any ringgit consequence.

Worked synthetic example — Shah Alam, Grade 40, through 9 Oct 2026

This is the case `selfcheck.py` locks in. **Synthetic.**

October 2026 starts on a Thursday. Available capacity from `_capacity_for` for SA / G40:

Dates	Rule	Available m ³ per day
Thu 1, Fri 2, Mon 5, Tue 6, Fri 9	Weekday 180 – 80	100
Sat 3	70 – 30	40
Sun 4	25 – 10	15
Wed 7, Thu 8	Maintenance 100 – 60	40

Sum of available capacity from 1 Oct through 9 Oct = 5×100 + 40 + 15 + 40 + 40 = **635 m³**.

Usable inventory = 145 – 80 = **65 m³**.

Dated supply = 635 + 65 = **700 m³**. `selfcheck.py` asserts this.

Demand with required date on or before 9 Oct:

Code	m ³
INT-MERDEKA	400
EXT-GAMUDA	300
EXT-JKR	180
EXT-SUNWAY	220
INT-ELMINA	150
EXT-YTL	160
Total	1,410

Dated gap = 1,410 – 700 = **710 m³**. `selfcheck.py` asserts this. The month still has capacity after 9 Oct, which is why a month-total view looks healthier than the week of the pour.

Capacity reduced by 30%. SUPPORTED as a scenario, not as a stored scenario. Set `capacity_factor` to 0.70. `apply_scenario` multiplies **available** capacity (not the raw daily capacity) by 0.70, optionally for one plant and one product. The linear programme is solved again. Inventory is unchanged unless an inventory adjustment is sent. The built-in Scenario B preset is not a 30% cut. It is **+15%** (`capacity_factor = 1.15`) on plant 1, product 1.

Bottleneck, in this prototype’s language. The plant-product whose unserved quantity is largest, and inside it the due date with the largest cumulative gap (`_crunch_date`). For the hero book that date is 2026-10-09. The code does not use the word bottleneck. Use “dated constraint” or “crunch date” so you stay faithful to the implementation.

9. DEMAND MODEL

Simple meaning

A demand line is one order that wants a quantity of one product at one plant by one date, and that already has a price for missing it.

Fields the engine actually uses

From `line_economics` and `_solve_lp`:

```
delay_cost_if_fully_unserved = delay_days_if_unserved × delay_cost_per_day

gross_consequence = contribution_margin + contractual_penalty + delay_cost_if_fully_unserved

confidence factor:
Confirmed = 1.00
Probable  = 0.75
Forecast  = 0.45
anything else falls back to 1.00 (confidence_factor)

expected_consequence = gross_consequence × confidence factor

unit_expected_rm = expected_consequence / requested_quantity
```

If the solver leaves unserved cubic metres:

```
fraction = unserved / requested_quantity

margin at risk = contribution_margin × fraction
penalty at risk = contractual_penalty × fraction
delay cost     = delay_cost_if_fully_unserved × fraction
expected cost  = expected_consequence × fraction

programme_days = delay_days × fraction    only when demand_type is Internal
                = 0                      otherwise
```

Source: `line_economics`, scaled, `consequence_for_unserved` in `economics.py`.

Hero unit rates (synthetic, full-quantity miss)

Order	Gross RM	Confidence	Expected RM	RM per m ³
Merdeka Podium, 400 m ³	16,000 + 0 + 3×20,000 = 76,000	1.00	76,000	190.00
Gamuda MRT Feeder, 300 m ³	18,000 + 30,000 + 0 = 48,000	1.00	48,000	160.00
JKR School Cluster, 180 m ³	5,400 + 22,000 = 27,400	1.00	27,400	152.22
Elmina, 150 m ³	4,500 + 2×8,000 = 20,500	0.75	15,375	102.50
Sunway, 220 m ³	11,000 + 8,000 = 19,000	1.00	19,000	86.36
YTL, 160 m ³	9,600	0.45	4,320	27.00
Mitrajaya, 180 m ³ , due 24 Oct	9,000 + 6,000 = 15,000	1.00	15,000	83.33

Mitrajaya's unit rate is lower than Gamuda's, but its date is after the crunch, so it does not compete for the 9 Oct window. Later capacity can serve it. `selfcheck.py` asserts Mitrajaya is fully served and that the tradeoff text does not mention Mitrajaya.

Criticality. Recorded for the planner. The base objective does not multiply by Critical 1.5 or Low 0.7. Those multipliers live in `CRITICALITY_MULT`. They are applied only inside `apply_scenario`, and only when criticality changes and the user did **not** also type a new delay cost. Then:

```
new delay_cost_per_day = old delay_cost_per_day × (new multiplier / old multiplier)
```

If you say “the optimiser prioritises critical projects”, that is **incorrect**. It prioritises the delay cost already stored on the row. Criticality is a label, unless a scenario uses it to rescale that cost.

Confirmed quantity and demand status. Stored, filtered, and displayed. They do not change the objective. Confidence level does.

How demand enters the engine. `solve_bucket` filters demands to one plant and product, drops non-positive quantities, attaches economics, then creates solver variables for each remaining id.

10. ALLOCATION ENGINE

Source file: backend/app/engine.py

Functions: allocate, solve_bucket, _solve_lp, _greedy, feasibility, score_targets

Library: PuLP (import pulp), solver PULP_CBC_CMD (CBC). Status must be Optimal or the function raises.

Simple explanation

Imagine one product at one plant. Every order has a required date and a cost per cubic metre of leaving it short. The computer hands out production days and usable stock. It is not allowed to use a day after the pour, to use another product, or to use more than the line can make. It keeps handing cubic metres to the shortage that hurts most, including splitting an order, until the supply in that window is gone. Then it writes down who was served, who was not, and how much money and programme delay that leaves.

People still have to accept it.

Technical explanation

The problem is solved **once per plant × product**. Six independent linear programmes, not one company-wide programme.

1. Decision variables

For each demand i in the bucket:

- $\text{unserved}_i \geq 0$ and $\leq$ requested quantity q_i
- $\text{inventory_use}_i \geq 0$
- $\text{production}_{i,t} \geq 0$ only for dates t where $t \leq \text{required_date}_i$

There is no binary “serve this order yes/no” variable. Quantities are continuous. An order can be split.

2. Objective function

```
Minimise
Σ over i of ( unit_expected_rm_i × unserved_i )
+ 0.0001 × Σ over all production variables
```

Source: the problem += statement in _solve_lp.

$\text{unit_expected_rm}_i$ is $\text{expected_consequence}_i / q_i$ from line_economics.

3. Objective coefficients

The coefficient on unserved_i is the expected RM per m^3 . The coefficient on each production variable is $1\text{e-}4$. The coefficient on inventory use is 0, which is why inventory is preferred to production when both can serve the same order. Internal versus external does not appear.

4–9. Constraints

Balance (demand satisfaction, with shortage allowed):

$$\sum_{\{t \leq \text{due}_i\}} \text{production}_{i,t} + \text{inventory_use}_i + \text{unserved}_i = q_i$$

Capacity, for each day t :

$$\sum \text{over orders } i \text{ that are allowed to use day } t \text{ of } \text{production}_{i,t} \leq \text{available_capacity}_t$$

Inventory:

$$\sum \text{over } i \text{ of } \text{inventory_use}_i \leq \text{usable_inventory}$$

Bounds:

```
production ≥ 0
inventory_use ≥ 0
0 ≤ unserved ≤ q
```

There is no constraint that forces an internal order to be served, no fairness constraint, no minimum service level, and no changeover constraint.

Negative inventory cannot occur: inventory variables are non-negative and their sum cannot exceed usable stock. Safety stock never enters the right-hand side.

Partial fulfilment is the usual result. Whatever is not produced or taken from inventory lands in `unserved`. Money is then scaled by `unserved / q` after the solve, in `consequence_for_unserved`. The solver itself minimises the linear unit rate times unserved, which matches that scaling because the rate is constant.

10–12. Solver output to recommendation

CBC returns variable values. `_solve_1p` reads them, snaps dust below 0.05 m^3 , and stores allocated, unserved, from_inventory, from_production, and production by day. `solve_bucket` turns that into line views, cards, policy comparisons, expedite rows, and paragraphs. `POST /api/allocate` returns the whole structure. The Allocation page renders it. Nothing is written to `decisions` until the person clicks Record.

What “minimising business consequence” means here

It means minimising the confidence-weighted sum of margin, penalty, and delay cost on the unserved fraction. It does not mean maximising profit, maximising service level, or minimising unserved cubic metres. Two plans with the same unserved m^3 can have very different objectives. The hero plan leaves 710 m^3 unserved in that window and still prefers that pattern because the unserved cubic metres are the cheaper ones.

How competing demands are compared

Inside one date window, compare `unit_expected_rm`. The narrative picks the served order with the **lowest** unit rate in the window (the marginal cubic metre) and the unserved order with the **highest** unit rate, excluding the same order compared with itself. The gap between those two rates is the cost of swapping 1 m^3 . On the hero book that sentence is Gamuda at $\text{RM}160/\text{m}^3$ versus JKR at about $\text{RM}152/\text{m}^3$, a difference of about $\text{RM}8$. `selfcheck.py` asserts the tradeoff text contains Gamuda, JKR, and $\text{RM}8$, and does not contain Mitrajaya.

Similar consequences

If two unit rates are equal, CBC may split the marginal cubic metres either way. The problem does not add a tie-break for internal work. The greedy earliest-date rule breaks ties by higher unit expected RM, then by id. Say that the linear programme is not required to prefer one of two equal rates.

Hero result the self-check locks (synthetic)

By 9 Oct, supply 700 m^3 is allocated as:

- Merdeka 400 served
- Gamuda 300 served
- JKR, Sunway, Elmina, and YTL unserved
- Mitrajaya, due 24 Oct, served from later capacity

Expected consequence on the unserved hero lines:

- JKR 27,400
- Sunway 19,000
- Elmina 15,375

- YTL 4,320
- **Total 66,095**

That 66,095 is the expected consequence of this one bucket under the recommendation, calculated from the formulas above. It is not a company result.

`selfcheck.py` also asserts two other synthetic outcomes: Penang LRT Depot is fully served and IJM Elevated Station is unserved (Shah Alam Grade 50); ECRL Station Precast is fully served and SP Setia Facade Panels are unserved (Pasir Gudang precast). The reason is the same unit-rate logic. Penang's delay cost per m³ is higher than IJM's margin-plus-penalty per m³. ECRL's six programme days at RM12,000 dominate Setia's margin and penalty per m³.

The three comparison policies

`_greedy` is not an LP. It sorts, then fills from inventory, then walks days from the start of the month taking whatever capacity remains.

- `internal`: internal rows first, then higher unit expected RM, then earlier date, then id
- `external`: external rows first, then the same
- `earliest`: earlier required date first, then higher unit expected RM, then id

Value protected versus a policy = that policy's expected consequence – the LP's expected consequence. `allocate()` sums this across buckets. On buckets where the greedy tie-break matches the LP, the difference is zero. The hero book is where the difference appears.

11. WHY LINEAR PROGRAMMING?

Why LP was selected. The decision is “how many cubic metres”, the cost of an unserved cubic metre is constant, and the limits (daily capacity, stock, due dates) are linear. That is a linear programme. CBC can solve this size instantly. The choice is visible in the code: continuous variables, linear objective, linear constraints, no training set.

What LP solves here. A feasible split of scarce dated supply that minimises a stated cost. It does not discover the cost. The cost is an input.

Why a simple rule is not enough. Each rule ignores part of the objective or treats a constraint as a priority. Section 2 walks through the four failures. The app keeps the rules so a planner can see the gap, not because they are recommended.

Highest margin ignores penalty, delay, confidence, and the fact that a cheaper-per-cube order may be the only one that can use an early day.

Internal-first and external-first add a preference the business said it did not want. The objective comment in `engine.py` says internal work is not preferred and external work is not preferred.

Advantages of this deterministic optimisation.

- Same data, same answer, every time.
- The constraint that failed can be pointed at (the override message names the date and the cubic metres).
- You can explain a one-cubic-metre swap in ringgit.
- No historical allocations are required before it can run.

Limitations of this LP.

- Costs are treated as certain and linear. A real penalty might be zero until a threshold, then a lump sum. A lump-sum penalty that does not scale with quantity would want an integer (binary) variable. **The code scales every penalty with the unserved fraction.** That is a simplification.
- Continuous cubic metres can recommend 95.2 m³. A pour might need a full load. Not modelled.
- No uncertainty inside the solve. Confidence is a fixed weight, not a scenario tree.
- No changeovers, travel, quality, or crew constraints, so a mathematically feasible plan can still be operationally awkward.
- Six separate solves cannot trade a cubic metre between plants or products.

When another method is more appropriate. Do not claim the prototype should have used these.

Method	When it would earn its place	Status here
Weighted scoring	One planner, few orders, and you only need a ranked list rather than a dated schedule	The unit rate is already a score. The LP is what turns scores into a feasible schedule
Integer or mixed-integer programming	All-or-nothing orders, minimum pour sizes, changeovers, or lump-sum penalties	Not implemented. Would be the honest upgrade if the business cannot split orders
Simulation	Capacity or demand is a distribution and you want the probability of a shortage	Not implemented. Confidence weights are not a simulation
Machine learning	You have history and you need to predict demand, delay, or a penalty that is not yet on the order	Not implemented. See section 12
Reinforcement learning	A long sequence of messy operational decisions with a reward you can observe	Not appropriate for this one-month book. There is no reward history
Rule-based allocation	The commercial policy really is “internal first” and finance has accepted the cost	Implemented only as a comparison, via <code>_greedy</code>

12. AI STRATEGY

AI actually implemented

None, if “AI” means a learned model or an LLM.

What exists is a **rules-based stand-in**, described in the module docstring of `backend/app/extractor.py`: “a rules-based stand-in for OCR and document extraction.” `extract_demand` uses regular expressions and keyword lists. The UI buttons say “Simulate OCR” and “Load sample email”. The sample strings `SAMPLE_OCR` and `SAMPLE_EMAIL` are pasted text, not a scanned image and not an inbox.

The stand-in classifies Internal versus External, confidence, plant, product, quantity, date, margin, penalty, delay, and criticality. It returns steps and warnings. It does not save and it does not allocate. A person must press “Add to demand book”.

Call this **PARTIALLY IMPLEMENTED document intake**, not an OCR system and not an LLM.

AI proposed for future development

PLANNED / CONCEPTUAL. Reasonable next steps, none of which are in the repo:

- A real OCR or document model to read a photographed PO, replacing the regex.
- A classifier for messy emails, still writing a draft a person confirms.
- A forecasting model that proposes Probable and Forecast quantities, which this engine can already weight at 0.75 and 0.45 once the numbers exist.
- Anomaly checks: a margin per m³ far outside the plant’s history, a capacity day of zero, a duplicate PO number.

AI that should not be used

An LLM should not be the allocator. The allocation is a constrained numerical decision. An LLM does not enforce `production + inventory + unserved = quantity` unless a solver does. It can invent a penalty, hide a trade-off, and give a different answer tomorrow. The repository already separates the two jobs in the first step message inside `extract_demand`: “The allocation engine is not used here. This step only prepares a demand line for a person to accept.”

AI-assisted data preparation means turning a document into fields a human checks.

AI-driven decision-making would mean the model chooses who is left unserved. This project does the first only as rules, and refuses the second.

Human-in-the-loop is implemented for both intake (draft then save) and allocation (recommend, check feasibility, require a reason when modified, store both versions).

13. SCENARIO SIMULATION

IMPLEMENTED. `apply_scenario` edits a copy of the world. `POST /api/scenarios/compare` accepts 1 to 3 scenarios, calls `allocate` on each, and returns totals plus a delta of expected consequence versus the first scenario.

What the user can change.

- Capacity factor 0.5 to 1.5, optionally limited to one plant and one product. Above 1 is described as emergency capacity above the committed plan. Below 1 is a reduction in available capacity.
- On-hand delta in m³ for one plant-product. Usable stock is recomputed. It cannot fall below zero on-hand.
- On one selected demand: quantity, required date, margin, penalty, delay days, delay cost per day, criticality, confidence.

What stays fixed unless you change it. The other products, the safety stock, the base calendar shape, and every demand you did not patch. The database is not updated.

How results are compared. Each scenario is a full re-solve. The UI chart and table show unserved m³, programme days, and expected consequence, and the delta versus scenario A. Source: `frontend/src/pages/Scenarios.tsx`.

Presets the page loads.

Preset	What the code sets	What it is not
A · Baseline	Factor 1, no adjustments	
B · Shah Alam overtime	Factor 1.15 on plant 1, product 1 (Shah Alam Grade 40)	Not a 30% reduction
C · Merdeka delay cost removed	INT-MERDEKA delay cost per day set to 0	Criticality is not changed

A 30% capacity reduction is **supported** by typing factor 0.70. A demand increase is **supported** by setting `requested_quantity`. An inventory increase is **supported** by a positive `on_hand_delta`. They are not separate named presets.

What you should be able to predict.

- Raising Shah Alam Grade 40 available capacity gives the solver more dated supply. The next cubic metres go to the unserved order with the highest unit expected consequence that can use those days. On the hero ranking that is JKR (RM152/m³), not YTL (RM27/m³).
- Setting Merdeka’s delay cost per day to 0 drops Merdeka’s unit expected consequence from RM190 to $RM16,000 / 400 = RM40$. It then ranks below Gamuda, JKR, Elmina, and Sunway. The window is no longer given to Merdeka. That is the point of preset C: the internal project loses when its delay cost is removed, which is evidence the engine was not “internal first”.

Do not memorise a cubic-metre result for preset B or C unless you have just run it. The direction above follows from the unit rates and the code path.

14. HUMAN-IN-THE-LOOP DECISION

Why it recommends instead of executing. The objective uses assumed penalties, assumed delay rates, and a linear split of lump-sum-looking penalties. A plant manager may know a constraint the model does not have. The code therefore never writes the plan into the calendar.

Approval. If every edited quantity is within 0.1 m^3 of the recommendation, status is `approved`. The default reason text in the UI is "Accepted the recommendation." The API still requires a reason of at least 3 characters.

Override. Changing any line by more than 0.1 m^3 sets status to `modified` and requires at least 8 characters of reason. `feasibility` then checks, for every date, that the sum of edited allocations due on or before that date is no more than usable inventory plus capacity through that date (tolerance 0.2 m^3). If not, the API returns 400 and the button is disabled because the preview says it cannot be produced.

What is stored. UTC time, username, plant, product, full recommended JSON, full final JSON, reason, status, both consequences, both unserved totals.

System recommendation is the output of `_solve_lp` for the current book.

Final business decision is the `final_json` on the latest `decisions` row for that plant-product.

They differ when status is `modified`. Business Impact uses the final JSON only while the set of demand ids still matches. Otherwise it shows the recommendation again and explains why.

Important limit. Approving does not change the next recommendation. Open Allocation again and the engine recomputes. The audit trail is a record, not a lock on the plan.

15. FRONTEND / USER INTERFACE

Six routes in `frontend/src/App.tsx`. Shared planner name in the sidebar.

Control Tower — /

- **User.** Planner or a manager looking at where the month is short.
- **Decision.** Where to open an allocation. Not the allocation itself.
- **API.** `GET /api/control-tower`.
- **Data.** Totals from `allocate(None)`, hotspots for constrained buckets, last 8 decisions.
- **KPIs.** Total demand, available capacity, dated capacity gap, internal demand, external demand, inventory at risk (assumption), margin at risk, programme days, value protected versus earliest-date.
- **Chart.** Horizontal bars of expected consequence by rule. Lower is better.
- **Table.** Plant, product, crunch date, shortfall, expected consequence, link to Allocation.
- **Action.** Read, then open a hotspot. No edit on this page.

Demand Hub — /demand

- **User.** Whoever maintains the order book.
- **Decision.** Whether a drafted line is real enough to enter the book. Not who gets capacity.
- **API.** `GET /api/meta`, `GET /api/demands`, `POST /api/demands/extract`, `POST /api/demands`, `DELETE /api/demands/{id}`.
- **KPIs.** Quantity in the current filter, internal, external, margin in view.
- **Table.** The demand columns in section 5, including confirmed quantity.
- **Actions.** Filter. Click a row for margin, penalty, delay, and the plant/product constraint. Remove a row only if its code contains `-IN-`. Paste text, simulate OCR, load the sample email, edit the draft, add it.
- **After add.** The row is in SQLite. The page tells the user to run allocation again. Allocation does not auto-refresh from this page.

Capacity Planner — /capacity

- **User.** Planner checking a plant-product.
- **Decision.** Whether the shortage is a date problem. Supports the later allocation; it does not allocate.
- **API.** `GET /api/capacity?plant_id&product_id`.
- **KPIs.** On-hand and its assumption value, safety stock, usable inventory, plus the charts.
- **Charts.** Daily capacity, planned production, available capacity, demand due, and cumulative supply versus cumulative demand (`ComposedChart` in `Capacity.tsx`).
- **Table.** Constrained dates and orders left unserved by the optimiser.
- **Action.** Change plant and product. Read-only.

Allocation Decision — /allocation

- **User.** The planner named in the sidebar.
- **Decision.** Accept or change the quantities, with a reason.
- **API.** `POST /api/allocate` with a baseline body, `POST /api/allocate/override` while editing, `POST /api/decisions` to record, `GET /api/decisions` for the trail.
- **KPIs.** Available supply, total demand, shortfall, expected consequence, programme days, margin left open.
- **Chart.** Expected consequence by policy for this bucket.
- **Tables.** Recommendation cards, editable quantities, expedite screen, audit trail.
- **Stepper.** Data → Recommendation → Human review → Approve or modify → Recorded.
- **After record.** A row is inserted. Production is unchanged.

Scenario Simulator — /scenarios

- **User.** Planner testing a what-if before committing.
- **Decision.** Whether overtime, a cost change, or a stock change is worth taking. The scenario is not saved as the plan.
- **API.** GET /api/meta, POST /api/scenarios/compare.
- **Chart.** Expected consequence for up to three named scenarios.
- **Action.** Edit factors and one demand or one stock delta, then compare. Focus a plant-product to read the tradeoff text.

Business Impact — /impact

- **User.** Someone explaining the modelled benefit.
- **Decision.** Whether the recommendation beats earliest-date, and what a recorded override did to that number.
- **API.** GET /api/impact.
- **KPIs.** Inventory value, excess inventory, working capital (all assumptions), margin deferred in the pilot, programme days, emergency cost worth pricing (assumption).
- **Chart.** Grouped bars for margin, penalties, and programme delay across Baseline, Pilot, and Approved.
- **Tables.** Before/after metrics, reference policies (internal-first and external-first), inventory lines, expedite actions, decisions.
- **Action.** Read-only.

Audit trail

Not its own page. It is the decisions table shown on Control Tower, Allocation, and Business Impact. That is **IMPLEMENTED**.

16. BUSINESS IMPACT

Every figure in this section is **estimated or modelled from synthetic inputs**. None of it is an observed Chin Hin result. There is no before/after in a live plant.

Metric	Formula in code	Source fields	Actual, estimated, or modelled	Limitation
Margin at risk	<code>contribution_margin × unserved / requested</code>	Demand row	Modelled	Treats margin as proportional. No price list
Penalty at risk	<code>contractual_penalty × fraction</code>	Demand row	Modelled	A real contract may be all-or-nothing
Delay cost	<code>(delay_days × delay_cost_per_day) × fraction</code>	Demand row	Modelled	Days scale linearly with the short fraction
Programme days	That delay-day product, internal only	Demand row	Modelled	Not a critical-path calculation
Expected consequence	Gross × confidence factor, then × fraction	<code>CONFIDENCE_FACTOR</code>	Modelled	The 0.75 and 0.45 weights are choices, not fitted probabilities
Gross consequence	Same without the confidence weight	Demand row	Modelled	Shown so a confirmed-order view is still available
Value protected	Earliest-date expected consequence – LP expected consequence	Both policies on the same synthetic supply	Modelled comparison	Only as good as the earliest-date baseline. It is not cash saved
Unserved m ³	Sum of unserved after the solve or after the human edit	Solver or decision JSON	Modelled quantity	Not weighed product that failed to ship
Inventory value	<code>on_hand × inventory_value_per_m3</code>	Product assumption	ASSUMED	Flagged in the UI
Excess inventory	<code>max(0, on_hand – safety – demand due by 2026-10-14)</code>	Hard-coded date in <code>_inventory</code>	ASSUMED definition of “14-day”	The date is written as a string, not “as-of + 14 days” in a general way
Working capital	Set equal to inventory value	Same assumption	ASSUMED	Ignores receivables, payables, and cash
Inventory at risk	On-hand × unit value, summed only for constrained buckets	Control Tower and impact	ASSUMED	Includes safety stock. The note in <code>control_tower</code> says so. It is not the value that will be scrapped
Emergency cost	<code>emergency_cost_per_m3 × unserved</code>	Product assumption	ASSUMED	Not in the LP. Worth expediting if unit consequence exceeds that cost by more than RM0.01. Confirmed orders use gross unit consequence; others use expected
Net expedite benefit	Consequence accepted – expedite cost	Those two	Modelled, assumption-based	A person would still have to approve overtime or outside supply

How to speak about ROI. You can say: on this synthetic book, the priced consequence of the recommendation is lower than the priced consequence of earliest-date planning, by `value_protected_vs_earliest_rm`. You cannot say the project saved that money at Chin Hin. To prove it later you would need the real book, the allocation that was actually produced, and the penalties and delay costs that were actually incurred.

17. ASSUMPTIONS

Assumption	Why it is needed	Value in the prototype	Where it lives	Class	If it is wrong	How to validate later
The book is a fair illustration, not a fact	A demo must run offline	2 plants, 3 products, 18 orders, Oct 2026	seed.py	Synthetic	Every conclusion about “the” plant is void	Replace with an ERP extract
Available capacity	Defines what this book may use	daily – planned, floor at 0	load_world	Model rule	Shortage is overstated or understated	Reconcile planned production to the live schedule
Safety stock is untouchable	Stops the solver draining the reserve	usable = max(0, on-hand – safety)	load_world	Model rule	A planner who will break safety stock will see a false shortage	Ask the plant whether safety can be borrowed, and at what cost
No product or plant substitution	Keeps each LP small and realistic for concrete grades	One solve per plant-product	allocate	Model rule	A certified substitute would be ignored	Get the quality rules in writing
Partial fills scale money linearly	Lets the LP stay linear	$\text{fraction} \times \text{totals}$	consequence_for_model_rule	Model rule	Lump-sum penalties are mispriced	Read the actual contracts
Confidence weights	Stops a forecast displacing a PO	1.00 / 0.75 / 0.45	CONFIDENCE_FACTOR	Assumed	A 45% forecast that is actually firm gets too little capacity	Fit weights to how often each class becomes a pour, or replace them with probabilities the business accepts
Criticality is not a solver weight	Avoids double-counting delay	Multipliers used only if a scenario changes criticality	apply_scenario	Model rule	Users may think “Critical” already changed the solve	Show them the unit RM, which is what the solver sees
Programme days only on internal demand	External pain is coded as penalty	Zero days for External	consequence_for_model_rule	Model rule	A customer delay with no penalty is invisible	Put that pain in penalty or delay cost
Emergency cost	Expedite advice	G40 RM95, G50 RM110, PCS RM180 per m ³ , flag = 1	PRODUCTS	Assumed	Expedite advice flips	Quote overtime and bought-in concrete
Inventory unit value	Stock KPIs	G40 RM280, G50 RM340, PCS RM1,200 per m ³	PRODUCTS	Assumed	Working-capital story is wrong	Use standard cost from finance
Working capital = inventory value	A single KPI the page can show	Equal to on-hand value	build_impact	Assumed	Finance will reject it	Use the real working-capital definition
Excess = on-hand – safety – demand through 14 Oct 2026	Flags stock above near-term demand	Date string 2026-10-14	_inventory	Assumed	Wrong horizon for “excess”	Make the window a parameter
1e-4 production penalty	Prefer stock over the line when the RM outcome is identical	1e-4	_solve_lp	Numerical	No business effect unless consequences are tiny	Leave it; state that it is a tie-break
Snap tolerance 0.05 m ³	Hide solver dust	TOL = 0.05	engine.py	Numerical	A true 0.04 m ³ residue disappears	Acceptable at this scale
Override date check tolerance 0.2 m ³	Avoid false infeasibility from rounding	0.2 in feasibility	engine.py	Numerical	A slightly over-allocated day might pass	Tighten if loads are small

Assumption	Why it is needed	Value in the prototype	Where it lives	Class	If it is wrong	How to validate later
Modified reason ≥ 8 characters	Force a real sentence	8	<code>record_decision</code>	Process rule	People type "xxxxxxx"	Pair with a review, not just a length check
Horizon	Bound the prototype	2026-10-01 to 2026-10-30	<code>HORIZON_START</code> , <code>HORIZON_END</code>	Prototype limit	Orders outside the month cannot be entered	Drive the horizon from the planning cycle
Seed version skip	Don't wipe decisions on every restart	<code>2026-10-hero-1</code>	<code>seed</code>	Prototype rule	Stale data if you change the seed and forget to bump the version	Bump the version when the book changes
CORS allows every origin	Local demo	<code>allow_origins=["</code>	<code>main.py</code>	Deployment assumption	Unsafe if the API is exposed	Restrict origins and add authentication
Planner name	Audit label	Default A. Rahman in <code>localStorage</code>	<code>planner.tsx</code>	UI default	The audit name can be anything the browser sends	Replace with a login

The list in `ASSUMPTIONS` inside `economics.py` is the set the UI prints. The table above is the longer register, including numerical tolerances the UI does not list.

18. LIMITATIONS

Current limitation	Why it matters	What a production system would change
All master data is synthetic	You cannot claim a Chin Hin outcome	Load plants, orders, and the calendar from source systems
No authentication or roles	Anyone who can open the API can record a decision as any name	Company login, planner versus viewer
Approval does not drive production	The plant can ignore the record and the model will not know	Write a released plan back to the schedule, then freeze it
<code>confirmed_quantity</code> and <code>demand_status</code> are ignored by the solver	The grid shows columns the maths does not use	Either use them or remove them so the story stays honest
Penalties and delay scale linearly	A 10% miss is priced as 10% of the full penalty	Binary or step penalties via integer variables, after legal review
No transport, site receiving windows, or truck capacity	A feasible plant plan can still miss the pour	Add dispatch constraints or a second scheduling step
No changeovers or grade-sequence rules	The daily capacity number is treated as fully usable	Sequence-dependent setup in an MILP, or a capacity derate agreed with the plant
Maintenance is a fixed seed note	A breakdown on the morning of the pour is invisible	A capacity scenario, or a live downtime feed
No quality holds or failed batches	Supply is assumed good	Reduce available capacity or usable inventory when QC rejects
Forecast uncertainty is a single weight	You do not see a bad-case plan unless you type a scenario	A small set of demand scenarios, already possible manually, not automated
Contracts are one penalty number	Real LD clauses are more specific	Store the clause, not only the RM
Human behaviour	A planner can type a meaningless reason	Workflow and review. The length check is only a floor
CBC / linear relaxation	Cannot express “serve all or nothing”	MILP if the business requires it
SQLite on one laptop, CORS open, no refresh schedule	Fine for a prototype, not for a plant	A server database, locked-down API, timed extract
Separate LPs per plant-product	Cannot recommend moving an order to the other works	Only add that if the product can legally be made there
Reseed deletes decisions	<code>selfcheck.py</code> calls <code>seed(force=True)</code>	Do not point a self-check at a database you need to keep
Impact “14-day” window is a fixed date	Easy to mis-explain	Parameterise it
Regex intake will misread messy documents	A wrong margin becomes a wrong allocation if someone saves it	Human confirm is already required; a real extractor still needs that confirm
No automated test suite	Regressions in the objective can ship	Keep <code>selfcheck.py</code> , and add API tests around feasibility and the hero asserts

19. VALIDATION & TESTING

What exists. `backend/app/selfcheck.py` is a script, not `pytest`. Run from backend with `python -m app.selfcheck`. It forces a reseed, solves, and asserts:

- Shah Alam G40 crunch date is 2026-10-09, supply about 700, gap about 710.
- Merdeka about 400 served, Gamuda about 300 served.
- JKR, Sunway, Elmina, YTL unserved; Mitrajaya served.
- Optimised expected consequence is below earliest, internal-first, and external-first on that bucket.
- The tradeoff text mentions Gamuda and JKR and RM8, and does not mention Mitrajaya.
- Penang served and IJM unserved; ECRL served and Setia unserved.
- Dated shortfall is greater than the aggregate gap, horizon surplus is positive, and value protected versus earliest-date is positive.

If an assert fails, the script crashes. There are no unit tests for `line_economics`, no API tests, no frontend tests, and no scenario tests.

Say this in the interview. “I have a self-check that locks the hero allocation. I do not have a test suite.”

Tests worth adding before any real deployment.

1. `line_economics` unit rates for a confirmed, probable, and forecast order, including quantity zero.
 2. Programme days are zero for an external order even if delay days are filled in.
 3. Safety stock is never allocated when on-hand equals safety.
 4. An order cannot receive production after its required date.
 5. Two products at the same plant do not share capacity.
 6. An override that needs 750 m³ by a date with 700 m³ of supply is rejected.
 7. An override that moves 50 m³ from a served order to an unserved order inside the supply is accepted and priced.
 8. A modified decision with a 3-character reason is rejected; an approval of the recommendation is accepted.
 9. Changing Merdeka delay cost to zero changes who receives the 9 Oct window.
 10. Capacity factor 0.7 reduces supply and does not change the database.
 11. A demand line with a duplicate `demand_code` is rejected.
 12. `confirmed_quantity` greater than requested is rejected on create.
 13. After a new intake line is inserted, the previous decision is not applied on the Impact page if demand ids no longer match.
 14. Extractor warnings on a text with no quantity and no date, and no allocation side effect.
 15. Self-check still passes after a refactor of the narrative.
-

20. REAL-WORLD DEPLOYMENT

PLANNED / CONCEPTUAL. Nothing below is built.

Piece	How it could fit	What would have to change first
Excel	A planner export of orders and a calendar, loaded by a replacement for <code>seed()</code>	Column mapping, units, and a rejection report for bad rows
SharePoint	Store the source spreadsheets or signed-off decision PDFs	Not the solver. A connector and permissions
SQL Server or similar	Replace SQLite. The tables in <code>db.py</code> are already ordinary relational tables	Migrations, backups, and foreign keys on <code>decisions</code>
ERP	Plants, items, open production orders, on-hand, safety stock, sales orders	This is the real data. The LP should sit beside the ERP, not inside a laptop file
Power BI	Read the decision and impact tables for management	Only after the numbers are real. A Power BI report of synthetic data is still a prototype
Power Automate	Notify a planner when a bucket becomes constrained, or when a decision is recorded	The API would need a stable identity and an event
API	Already the boundary (<code>main.py</code>)	Authentication, HTTPS, a restricted CORS list, logging
Data refresh	A scheduled job that reloads capacity and demand and then re-solves	A rule for what happens to an approved plan when the book changes. Today, impact silently falls back
User roles	Planner records; manager reads; admin reseeds	There is no user table
Audit	The <code>decisions</code> table is the start	Add user id from login, and do not trust a name typed in the browser

Before a real company deployment: replace synthetic data, validate every assumption in section 17 with plant and finance, decide whether partial fills and linear penalties are acceptable, add the missing physical constraints the plant actually hits, stop the solver from being the system of record for production, and lock the API down.

21. BUSINESS REFERENCES & METHODOLOGY

What the repository itself cites. Nothing. There is no bibliography in the code or the README. Do not invent a citation.

How the implemented process maps onto standard planning language. This mapping is teaching context, not something the code claims.

Planning idea	Where it shows up in this prototype
Demand planning	One book of firm, probable, and forecast lines. No forecast algorithm
Rough-cut capacity planning	Daily capacity versus demand by plant and product, before a detailed sequence
Master scheduling / finite loading	The LP assigns quantities to days on or before the due date, up to available capacity
Inventory policy	Safety stock is reserved; only the surplus is allocatable
S&OP, in miniature	Scenarios change capacity, stock, and commercial terms and compare the consequence. There is no monthly S&OP meeting workflow
Manufacturing planning and control	The prototype covers the allocation decision. It does not cover shop-floor execution, costing, procurement, or distribution

Recommended reading, clearly external to the project. These are standard public references for you to study. They are not sources of the numbers in the app.

- Vollmann, Berry, Whybark, Jacobs, *Manufacturing Planning and Control for Supply Chain Management* — the usual textbook frame for demand, master scheduling, and capacity.
- Chopra, *Supply Chain Management: Strategy, Planning, and Operation* — inventory and supply constraints.
- Taha, *Operations Research*, or any standard LP chapter — formulation, feasible region, objective.
- PuLP documentation for the library that actually builds the model in `_solve_lp`.

If an interviewer asks “which paper is this based on?”, the honest answer is: it is a linear programme written for this prototype, not an implementation of a published Chin Hin paper.

22. WHY THIS PROJECT IS NOT JUST A DASHBOARD

A traditional dashboard. Data is queried and drawn. The person looks at a chart and decides elsewhere, with no record of the alternative they rejected.

This repository.

```
Synthetic book in SQLite
→ one demand table for internal and external
→ available capacity and usable inventory
→ cumulative gap by date
→ linear programme per plant-product
→ recommendation, with the RM reason
→ comparison with three rules
→ human edit, rejected if infeasible
→ stored decision (recommended JSON and final JSON)
→ modelled impact versus earliest-date
```

The distinction matters for a portfolio because the hard part is the decision model and its constraints, not the bar chart. If you only describe the screens, a reviewer will correctly call it a dashboard. If you can write the objective, the balance constraint, and the reason a higher-margin order can still lose, you are describing an operations-research decision tool with a review step.

23. INTERVIEW CHEAT SHEET

For each question: a short answer you can say, then the deeper point, then the file.

Basic

1. What problem are you solving?

Short: When a plant cannot meet every order by its required date, which orders should get the cubic metres, and what does that cost?

Deeper: The month can still be in surplus. The shortage is dated.

File: `control_tower()` in `engine.py`.

2. What data did you use?

Short: A synthetic October 2026 book: two plants, three products, a daily calendar, opening stock, and 18 orders. It is not Chin Hin data.

Deeper: Plus any line or decision created during the demo, still inside the prototype.

File: `seed.py`, `db.py`.

3. Who uses the system?

Short: A planner who reviews the recommendation and records their name against the decision.

Deeper: The name is typed in the browser. There is no login.

File: `planner.tsx`, `record_decision`.

4. Why did you build it?

Short: A chart of the shortage does not choose who absorbs it, and the obvious priority rules disagree with the priced consequence.

Deeper: The self-check shows the linear programme beating earliest-date, internal-first, and external-first on the hero book.

File: `selfcheck.py`.

Intermediate

5. How does the allocation engine work?

Short: For each plant and product, minimise the expected cost of what you leave unserved, using only stock above safety and capacity on or before each order's date.

Deeper: Section 10. Continuous quantities, partial fills, CBC.

File: `_solve_lp`.

6. Why linear programming?

Short: The cost per unserved cubic metre is constant and the limits are linear, so an LP gives a repeatable feasible plan.

Deeper: A rule cannot see the whole window at once. An LLM should not be the solver.

File: `engine.py` module docstring.

7. How did you calculate capacity?

Short: Available capacity is daily capacity minus production already committed. Usable stock is on-hand minus safety stock. Dated supply adds those up only through the required date.

Deeper: The hero date is exactly $635 + 65 = 700$ against 1,410 due.

File: `load_world`, `_capacity_for`, `selfcheck.py`.

8. How did you handle internal versus external demand?

Short: They share one table and one objective. Internal lines also carry programme days. Neither type gets a bonus.

Deeper: The greedy policies exist to show what a preference would cost.

File: `demands.demand_type`, `_greedy`, `consequence_for_unserved`.

9. How did you measure business impact?

Short: Compare the priced consequence of earliest-date planning with the priced consequence of the recommendation, on the same synthetic supply.

Deeper: That gap is modelled value protected, not observed savings. Inventory and expedite numbers are

assumptions.

File: `impact.py`.

Advanced

10. What is your objective function?

Short: Minimise the sum of unit expected RM times unserved cubic metres, plus a tiny weight on production so stock is used first.

Deeper: Unit expected RM is $(\text{margin} + \text{penalty} + \text{delay days} \times \text{cost per day}) \times \text{confidence}$, divided by quantity.

File: `_solve_lp`, `line_economics`.

11. What are your decision variables?

Short: For each order, how much comes from stock, how much is made on each allowed day, and how much is left unserved.

Deeper: Production variables are not created for days after the required date.

File: `_solve_lp`.

12. What are your constraints?

Short: Production plus stock plus unserved equals the order quantity. Each day cannot exceed available capacity.

Total stock used cannot exceed usable inventory.

Deeper: No cross-plant constraint, because those are separate solves.

File: `_solve_lp`, `allocate`.

13. Why not machine learning?

Short: The problem is a constrained allocation with known costs, not a prediction from history. There is no historical allocation data in the project.

Deeper: ML would help later by forecasting demand or estimating a missing penalty. It would still feed the solver.

File: absence of any model file; `extractor.py` is regex.

14. How would you validate the model?

Short: Lock the hero case with the self-check, then sit with a planner and test whether the penalties, the safety stock, and the "no substitution" rule match the plant.

Deeper: Section 19. Also shadow-run against the allocation the plant actually made.

File: `selfcheck.py`.

15. How would you handle uncertain demand?

Short: The prototype already weights Confirmed, Probable, and Forecast, and the scenario page can change a quantity or a confidence and re-solve.

Deeper: That is not a stochastic programme. A production version would run a few explicit demand cases and show the spread.

File: `CONFIDENCE_FACTOR`, `apply_scenario`.

Challenging questions

16. "Isn't this just a dashboard?"

Short: The charts explain a linear programme, a feasibility check, and a stored override. A dashboard does not do those three.

Deeper: Section 22.

File: `_solve_lp`, `feasibility`, `decisions`.

17. "Why should I trust your recommendation?"

Short: Trust the inputs first. Given those inputs, the recommendation is the feasible plan with the lowest expected consequence, and you can see the one-cubic-metre swap that justifies it.

Deeper: If the penalty is wrong, the recommendation is wrong. That is why a person approves it.

File: `_narrative`, `ASSUMPTIONS`.

18. "What if the plant manager disagrees?"

Short: They edit the quantities. If the edit cannot be made by the due date, the system refuses it. If it can, they record a reason and the audit trail keeps both versions.

Deeper: The disagreement does not retrain a model. It is a business override.

File: `feasibility`, `record_decision`.

19. “What if your data is wrong?”

Short: Then the allocation is wrong. The prototype’s data is synthetic, so I would not use it to schedule a real pour.

Deeper: The dangerous fields are required date, quantity, penalty, and delay cost, because they enter the objective or the constraints directly. Criticality by itself does not.

File: `line_economics` versus `project_criticality`.

20. “Why shouldn’t the highest-margin customer get the capacity?”

Short: Because the cost of a miss includes the penalty and the programme delay, and because a high-margin order due later cannot use capacity that expires at an earlier pour.

Deeper: JKR’s margin is small and its penalty is not. Merdeka’s margin is not the largest term on that row; the delay cost is.

File: `line_economics`, hero rows in `seed.py`.

21. “Why not prioritise internal projects?”

Short: An internal project is not automatically the more expensive miss. The comparison policy is in the app, and on the hero book it leaves a higher expected consequence than the solver.

Deeper: Removing Merdeka’s delay cost in a scenario is the clean demonstration that the internal label was not what protected it.

File: `_greedy`, Scenario preset C.

22. “How do you know your model actually saves money?”

Short: I don’t, not on real operations. I know that on this synthetic book the priced consequence is lower than three stated rules.

Deeper: Proving savings needs the live book and the incurred penalties.

File: `value_protected_note` in `impact.py`.

23. “How would you deploy this at Chin Hin?”

Short: I would not deploy this database. I would keep the formulation, replace the seed with ERP orders and the real calendar, validate the cost rules with planning and finance, and add login plus a write-back only after a person approves.

Deeper: Section 20. Also say the current repo does not contain Chin Hin data.

File: `seed.py` versus the missing connector.

24. “What would you improve with another 12 weeks?”

Short: Real data, contract-true penalties, plant constraints the managers name (changeover, trucks, minimum pour), tests around the solver, and an approval that freezes a plan instead of only logging it.

Deeper: Do not spend the 12 weeks on a better chart or on an LLM allocator.

File: sections 18 and 19.

24. PROJECT EXPLANATION TEMPLATES

30-second explanation

Use the one in section 1.

1-minute explanation

Use the one in section 1.

3-minute explanation

Use the one in section 1. If you are stopped early, stop after the objective and the human approval. Do not open with the technology stack.

Technical explanation

"allocate loads SQLite, derives available capacity and usable inventory, and calls `solve_bucket` for each plant and product. `_solve_lp` builds a PuLP minimisation: unserved cubic metres are penalised at the order's expected ringgit per m³, production on each feasible day and inventory use must sum with unserved to the requested quantity, daily production cannot exceed available capacity, and inventory use cannot exceed stock above safety. CBC solves it. `_greedy` prices three priority rules on the same supply. `feasibility` rejects an override that breaks the dated supply. `record_decision` stores both JSON payloads."

Business explanation

"We have two kinds of demand and not enough dated supply. I price the miss, I let a solver assign the scarce cubic metres, and I show the planner what their usual rules would have cost. They can override. I do not pretend the synthetic book is the company's result."

Data Analyst explanation

"The fact I care about is the demand line. Plant, product, and date are the grains. I had to stop people adding the month up and calling it a surplus. The measure that changes the decision is expected ringgit per unserved cubic metre, not the raw margin. Several columns on the demand extract — confirmed quantity, status, criticality — are visible and are not in the calculation, and I should say that before someone builds a report on the wrong field."

Data Scientist explanation

"I did not train a model. The structure is an LP because the decision is continuous, the objective is linear, and there is no labelled history. Uncertainty is a fixed confidence weight plus manual scenarios. The document step is regular expressions standing in for extraction, with a human confirm. If I added ML, it would estimate demand or a missing cost and pass numbers into this solver. It would not replace CBC."

Business Analyst explanation

"The user is the planner. The decision is the allocation under a shortage. The process is book, capacity, recommendation, review, record. I separated assumptions — inventory value, emergency cost, confidence weights — from the identity of the plants. Success in the prototype is a lower modelled consequence than earliest-date, plus an audit trail. Success in production would be those same comparisons on live orders, after the plant agrees the constraints."

25. “IF THE INTERVIEWER ASKS WHY...”

Why this problem? Because the expensive mistake is not failing to draw the shortage. It is giving the scarce cubic metres to the wrong order.

Why these datasets? Because the solver needs a where (plant, product), a when (calendar and required date), a how much (quantity, stock, capacity), and a cost of missing (margin, penalty, delay, confidence). Anything that does not enter those four jobs was left out or left unused.

Why these KPIs? Dated shortfall, expected consequence, programme days, and value protected versus a named baseline. Inventory value is on the page because working capital was in the business question, and it is labelled as an assumption so it is not confused with the optimiser.

Why this architecture? The browser should not ship a solver. FastAPI exposes the decision. SQLite holds the book. PuLP holds the maths. The split is visible in the proxy and in `main.py`.

Why Python? The optimiser and the data load are Python. PuLP is a Python library. The API is thin.

Why FastAPI? It gives typed request bodies — the scenario factor limits, the confidence literals — and a small route list. There is no deeper framework in the repo.

Why SQLite? One file, no database server, enough for foreign keys and a demo. It is a prototype choice, not a plant standard. The path is `backend/data/cdi.db`.

Why linear programming? Section 11.

Why not machine learning? Section 12 and question 13.

Why not an LLM? It would not enforce the capacity constraint, and the project already isolates text-to-fields from the allocator.

Why human approval? The costs are assumptions and the plant knows constraints that are not in the model. The audit trail is the evidence of who chose.

Why scenario analysis? So a planner can see the allocation move when capacity, stock, or a delay cost changes, before they treat the baseline as fate.

Why synthetic data? There is no live extract in the project. The book was built so that a dated shortage and a month surplus exist at the same time, which is the point worth demonstrating. It must be described as synthetic every time.

Why this project matters. It is a worked example of turning a planning argument — “we should serve internal first” or “serve the best customer” — into a constraint, an objective, and a number the planner can override.

26. MY PROJECT STORY

The problem starts when a building-materials plant accepts both its own projects and outside customers onto the same line. A monthly total can say there is enough concrete. The pour dates say otherwise, because a cubic metre made after the required date cannot go back in time.

I put both kinds of demand in one book rather than two queues. Each line has a plant, a product, a required date, a quantity, and a cost of missing it: contribution margin, a contractual penalty, and, for internal work, programme days times a daily delay cost. Confidence scales that cost so a forecast does not look as solid as a purchase order. I did not give internal work a bonus and I did not give external work a bonus.

The supply side is a 30-day calendar. Available capacity is what remains after production already committed outside the book. Usable inventory is what remains after safety stock. I generated that book in code so the prototype would open on a real constraint: at Shah Alam, Grade 40, demand due by 9 October 2026 is 1,410 m³ and dated supply is 700 m³. That book is synthetic. It is not a Chin Hin extract.

The model is a linear programme, solved separately for each plant and product with PuLP and CBC. It chooses how much of each order to leave unserved. It cannot exceed the day, cannot use stock below safety, and cannot produce after the due date. The objective is the expected ringgit on the unserved part. I also run three priority rules on the same supply so I can say, in money, what "earliest first", "internal first", and "external first" give up.

The application walks a planner from the hotspot, to the reason, to an editable quantity. If their edit cannot be produced, it is rejected. If it can, both the recommendation and their decision are stored with their name, the time, and the reason. A scenario page reruns the same engine. An impact page compares the earliest-date rule, the recommendation, and the recorded decision, and it labels inventory value and emergency cost as assumptions.

I check the hero result with a self-check script. I do not yet have a full test suite. The prototype does not schedule trucks, changeovers, or a live ERP, and approving a plan does not release production. The next serious step is to replace the synthetic book with real orders and to confirm, with the plant and with finance, that the penalty and the delay cost are the consequences they are actually willing to manage.

27. FINAL KNOWLEDGE CHECK

CAN I DEFEND THIS PROJECT?

- [] I can explain the business problem as a dated shortage, not only as “demand exceeds capacity”.
- [] I can say, unprompted, that the data is synthetic and is not Chin Hin operational data.
- [] I can name the real tables: plants, products, capacity calendar, inventory, demands, decisions, meta.
- [] I can say which “datasets” do not exist: delivery, actual production, a separate project schedule.
- [] I can draw plant → capacity, plant+product → inventory, plant+product → many demands, decisions as a JSON audit copy.
- [] I can write available capacity, usable inventory, and dated supply from memory.
- [] I can compute the hero 700 m³ and 710 m³ gap without looking it up.
- [] I can write gross consequence and expected consequence, including the three confidence weights.
- [] I can explain that criticality, confirmed quantity, and demand status are not solver inputs.
- [] I can state the objective, the three families of variables, and the three constraints.
- [] I can explain the 0.0001 production term without pretending it changes who is served.
- [] I can explain why partial fulfilment is allowed and how money scales.
- [] I can explain why Merdeka and Gamuda are served and JKR is not, using RM per m³ and the due date.
- [] I can explain why earliest-first, internal-first, external-first, and highest-margin-first can all be wrong.
- [] I can distinguish the linear programme from the greedy comparison policies.
- [] I can say CBC via PuLP, once per plant-product.
- [] I can describe what an infeasible override looks like.
- [] I can distinguish the recommendation from the recorded decision.
- [] I can say that recording a decision does not update the production calendar.
- [] I can describe the scenario levers: capacity factor 0.5–1.5, stock delta, and one demand’s commercial fields.
- [] I can predict the direction of “remove Merdeka’s delay cost” without claiming a memorised cubic-metre printout.
- [] I can say there is no LLM and no trained model; intake is regex and must be confirmed.
- [] I can separate modelled value protected from actual savings.
- [] I can list inventory value, emergency cost, confidence weights, and working capital as assumptions.
- [] I can name the self-check and admit there is no test suite.
- [] I can walk all six screens and say which decision each one supports.
- [] I can list the constraints a real plant would add before go-live.
- [] I can tell the project story in section 26 without reading it.